TCC - Anotações

Projeto: Óculos inteligentes para detecção de objetos perigosos e descrição por áudio em ambientes escolares

1. Escopo do Projeto

- Problema: Dificuldade de locomoção e segurança de deficientes visuais em corredores escolares (obstáculos dinâmicos e estáticos).
- Solução: Dispositivo vestível (óculos/boné) que utiliza Visão Computacional para identificar objetos e alertar o usuário via áudio em tempo real.
- Público-alvo: Estudantes com diferentes graus de deficiência visual.

2. Arquitetura do Sistema

Hardware (Edge Computing)

- Processamento: Raspberry Pi (SBC - Single Board Computer).
- Entrada: Raspberry Pi Camera Module (Interface CSI) acoplada ao suporte frontal.
- Saída: Fone de ouvido (P2 ou Bluetooth) para feedback auditivo.
- Alimentação: Power Bank 5V/3A via USB-C.

Software e IA

- Linguagem: Python 3.
- Modelo de IA: YOLOv11 e Yolov26.
- Custom Dataset (Roboflow)
- Bibliotecas: ultralytics (YOLO), opencv-python (Câmera).

3. Cronograma e Status (2026)

Atividade	Prazo	Status
Proposta Inicial da Pesquisa	16/03	✅ Concluído
Levantamento Bibliográfico	Março	✅ Concluído
Treinamento do Dataset Customizado (Roboflow)	Março	✅ Concluído
Comparação de Modelos	Abril	✅ Concluído
Montagem do Protótipo (Raspberry + Câmera)	Maio	✅ Concluído
Integração de Áudio e Lógica de Software	Junho	✅ Concluído
Testes de Campo e Validação	Junho	✅ Concluído
Entrega da Documentação Final	30/06	✅ Concluído

4. Recursos e Links Úteis do Nosso Projeto

- Documentação: Proposta Inicial da Pesquisa
- Treinamento: Google Colab Notebook

Sumário

1. Cursos e Estudos do RaspBerry 4

    1.1. Raspberry 5

    1.2. Informações Elétricas que precisamos saber para montar o circuito!!! 11

2. Guia Prático: Configuração e Primeiro Projeto (Blink LED) 12

3. Visão Computacional: Dataset e Arquitetura YOLO 13

    3.1. Custom Dataset no RoboFlow 13

    3.2. YOLO 15

4. Comparações de Modelos de IA 17

    4.1. Arquitetura (Teoria) 17

        4.1.1. Os "Rápidos e Diretos" (One-Stage Detectors) 17

        4.1.2. Os "Econômicos" (Mobile-First) 18

        4.1.3. Os "Pintores de Pixel" (Segmentação Semântica) 18

        4.1.4. Resumo para seus estudos 19

    4.2. Métricas (Comparação) 20

        4.2.1. Tabela de Comparação Reais 21

1. Cursos e Estudos do RaspBerry

Softwares Necessários

- Raspberry Pi Imager: Gravador oficial do Sistema Operacional.
- PuTTY: Cliente SSH para acesso remoto via terminal.
- VNC Viewer: Acesso remoto à interface gráfica.

Cursos e Tutoriais Recomendados

- Configuração Passo a Passo (Raspberry Pi em um PC)
- Curso de Raspberry Pi - Canal Curso em Vídeo
- Introdução ao Raspberry Pi - Canal CFB Cursos

1.1. Raspberry

O Raspberry Pi é um microcomputador de uma placa única SBC (Single Board Computer).

Componentes Principais da Placa

SoC – System on a Chip

No Raspberry Pi, quase tudo está dentro de um único chip (reduz tamanho, consumo de energia e custo), chamado SoC. Ele inclui:

- CPU → processador principal
  - É o "cérebro" do Raspberry
  - Responsável pela execução dos programas e do sistema operacional
  - Arquitetura ARM
  - Executa Linux, Python, C, Java etc.
- GPU → processamento gráfico e vídeo
  - Responsável por:
    - saída de vídeo
    - aceleração gráfica
    - decodificação de vídeos
  - Trabalha junto com HDMI e interfaces de vídeo
- Controlador de memória
  - Usada para armazenar dados e programas em execução
  - Soldada na placa (não expansível)
  - Pode variar (1GB, 2GB, 4GB, 8GB)
  - Compartilhada entre CPU e GPU
- Controladores de I/O

Armazenamento (Cartão microSD)

- Funciona como o HD/SSD
- Sem cartão SD, o Raspberry não inicia.
- Nele ficam:
  - Sistema operacional
  - Arquivos
  - Programas

Portas USB

- Permite conexão de:
  - teclado

    - mouse
    - pendrive
    - adaptadores
- Em modelos recentes: USB 2.0 e USB 3.0 (mais rápido)

CSI (Camera Serial Interface)

- Interface dedicada para câmera
- Usada para:
    - Raspberry Pi Camera Module
    - projetos de visão computacional
- Comunicação rápida e direta com o SoC

DSI (Display Serial Interface)

- Interface dedicada para telas
- Usada para:
    - display oficial do Raspberry
    - telas touchscreen
- Melhor desempenho que HDMI para displays pequenos

GPIO – General Purpose Input/Output

- Base de praticamente todo projeto eletrônico com Raspberry.
- 40 pinos
- Usados para:
    - entrada (sensores, botões)
    - saída (LEDs, motores, relés)
- Incluem:
    - GPIO digital
    - I2C
    - SPI
    - UART
    - pinos de energia (3.3V, 5V, GND)

Porta HDMI

- Saída de vídeo digital
- Conecta a:
    - monitor
    - TV
- Alguns modelos usam micro-HDMI

LEDs

- LEDs integrados na placa
- Indicam:
    - energia ligada
    - atividade do cartão SD
    - status do sistema

Porta de Alimentador

- Fonte de alimentação via:
    - USB-C ou Micro-USB
- Fornece energia para:
    - placa
    - portas USB
    - periféricos conectados
- Tensão: 5V
- Corrente: depende do modelo (geralmente 3A)
- Fonte fraca causa instabilidade.

Wi-Fi + Bluetooth

- Integrados no SoC ou em chip dedicado
- Permitem:
    - conexão com internet sem fio
    - comunicação com sensores e dispositivos Bluetooth
- Muito usados em projetos IoT

Porta Ethernet (Rede cabeada)

- Para conexão com a internet via cabo
- Mais estável que Wi-Fi

Componentes externos (usados com Raspberry)

Sensores

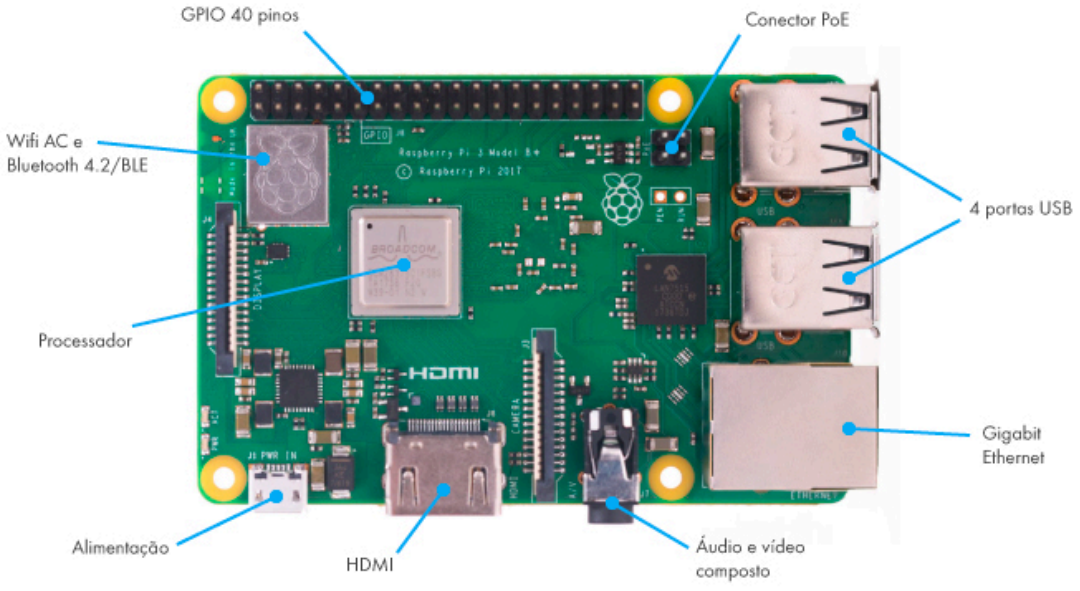
- Temperatura (DHT11, DHT22); Umidade; Movimento; Luz.

Atuadores

- LEDs; Motores; Servos; Relés.

Acessórios

- Case
    - Protege a placa contra poeira, curto-circuito, impactos
    - Evita tocar diretamente nos componentes
    - Dá organização ao projeto
- Cooler ou dissipador de calor
    - O SoC (CPU + GPU) esquenta bastante
    - O dissipador:
        - absorve o calor
        - melhora a dissipação térmica
    - Evita:
        - queda de desempenho (thermal throttling)
        - travamentos
    - redução da vida útil
- Camera Module
- Display LCD / Touch



Estrutura de software

Sistema Operacional (o que usamos)

- Raspberry Pi OS (Linux baseado em Debian)

Programação (o que usamos)

- Python → mais comum para GPIO e automação

Comandos CLI

- pwd: retorna o caminho atual
- ls: lista todos as pastas ou arquivos que tem dentro do diretório atual
- ls -R: lista as pastas e subpastas, ou arquivos que existem dentro
- ls -a: lista os arquivos ocultos
- ls -al: lista os detalhes de tudo
- tree: lista todos diretórios/arquivos em formato de árvore
- tree -d: lista os diretórios em formato de árvore
- cd: funciona para se movimentar entre os diretórios
- cd ..: volta para o diretório anterior
- nano texto.txt : criar o arquivo texto, nano é um editor de arquivos do linux
- cat texto.txt: mostra o conteúdo do arquivo em precisar abrir ele
- mkdir teste: cria uma pasta
- cp origem destino: copiar um arquivo de um lugar de origem para o outro de destino
- mv origem destino: move um arquivo de um lugar de origem para o outro de destino
- rm: remove/exclui o arquivo
- rmdir: remove/exclui a patas
- pinout: mostra a estrutura do raspberry!!!!!!!!!
- ifconfig: IP do rasp
- sudo raspi-config: mostra a tela de configuração do rasp
- sudo reboot: reinicia o rasp (nao sei se perde as cs)
- sudo halt: desliga o rasp
- sudo shutdown now: desliga o rasp agora
- sudo shutdown -r 100: desliga o rasp daqui 100 segundos
- whereis putty: permite achar se existe o programa ou não

Gerenciamento de pacotes com o comando APT

APT (Advanced Package Tool) é o gerenciador de pacotes das distribuições baseadas em Debian, incluindo o Raspbian / Raspberry Pi OS.

- Instalar programas
- Atualizar o sistema
- Remover softwares
- Gerenciar dependências automaticamente

> Pense no APT como a “Play Store” do Linux, só que via terminal.

Comandos CLI

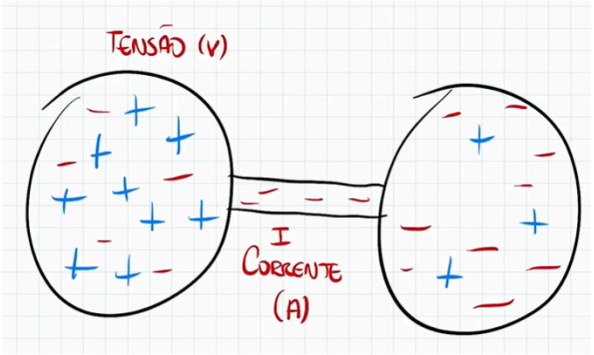
- sudo apt update: atualizar a lista de pacotes de versões disponíveis (sempre execute first)
- sudo apt upgrade: atualizar o sistema (confirmação automática: sudo apt upgrade -y)
- sudo apt full-upgrade: atualização completa (pode remover pacotes antigos)
- sudo apt install nome-do-pacote: instalar pacotes
- sudo apt install git curl htop: instalar vários de uma vez
- sudo apt remove nome-do-pacote: remover mantendo configurações
- sudo apt purge nome-do-pacote: remover completamente
- sudo apt autoremove: remover dependências não usadas
- sudo apt clean: limpar pacotes baixados
- apt search nome: buscar pacotes
- apt show nome-do-pacote: mostrar detalhes

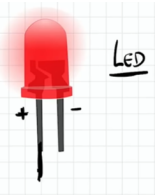
Onde ficam os pacotes?

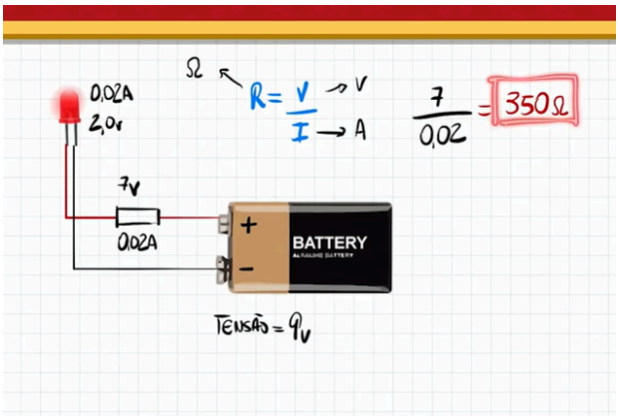
- Pacotes baixados: /var/cache/apt/archives/
- Lista de repositórios: /etc/apt/sources.list ou /etc/apt/sources.list.d/

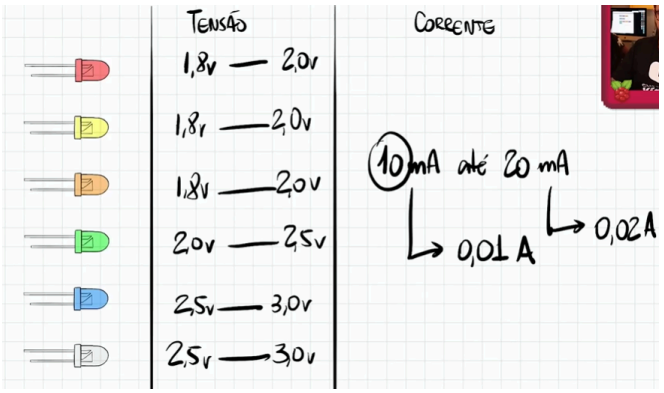
1.2. Informações Elétricas que precisamos saber para montar o circuito!!!

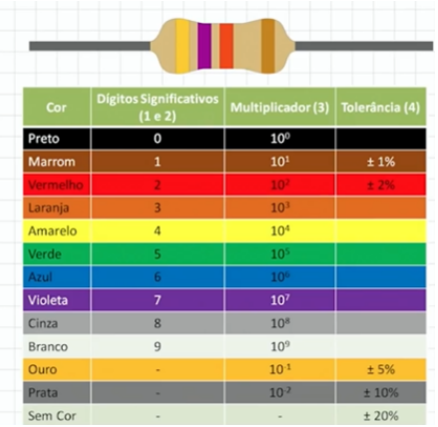












Cor	Dígitos Significativos (1 e 2)	Multiplicador (3)	Tolerância (4)
Preto	0	10^0	
Marrom	1	10^1	± 1%
Vermelho	2	10^2	± 2%
Laranja	3	10^3	
Amarelo	4	10^4	
Verde	5	10^5	
Azul	6	10^6	
Violeta	7	10^7	
Cinza	8	10^8	
Branco	9	10^9	
Ouro	-	10^{-1}	± 5%
Prata	-	10^{-2}	± 10%
Sem Cor	-	-	± 20%

- Menos que o mínimo, led nao acende, se colocar mais, queima

2. Guia Prático: Configuração e Primeiro Projeto (Blink LED)

Vídeos Base

- Parte 1: Configuração Inicial e Imager
- Parte 2: Instalação de Apps e Acesso Remoto

Passo a Passo

1° Instalar RaspBerry Pi Imager e Configurar o OS no cartão microSD

2° Instalar PuTTY (SSH): use para acessar o terminal via IP ou Hostname (aqui ocorre a conexão)

3° Instalar VNC Viewer: para ver a “área de trabalho” do RaspBerry no seu PC

4° Atualização de Segurança: no terminal, prepare o comando *sudo apt update* e *sudo apt upgrade*

5° Configurar o RaspBerry do seu jeito!!!

6° Montagem do circuito (smp com o raspberry desligado)

- Componentes: 1 LED, 1 Resistor, Jumpers.
- Conexão:
    - Anodo (perna longa do LED): Conectar ao Pino 12 (GPIO 18).
    - Catodo (perna curta): Conectar ao Resistor e este ao Pino 6 (GND).

7° Abra o Thonny IDE que é para Python e programe:

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BOARD)

pinoLed=12
cont=10

GPIO.setup(pinoLed,GPIO.OUT)

for i in range(cont):
   GPIO.output(pinoLed,GPIO.HIGH)
   time.sleep(1)
   GPIO.output(pinoLed,GPIO.LOW)
   time.sleep(1)

print("fim")
GPIO.cleanup()
```

3. Visão Computacional: Dataset e Arquitetura YOLO

3.1. Custom Dataset no RoboFlow

O Roboflow é uma plataforma de inteligência artificial focada em visão computacional — ou seja, ela permite que computadores “enxerguem” e entendam imagens e vídeos.

- Desenhar as Caixas: Você sobe suas fotos lá e usa o mouse para circular o que é um "extintor", o que é um "cone", etc. Sem isso, o YOLO não sabe o que olhar.
- Organizar o "Pacote": O YOLO exige que as fotos e os textos com as coordenadas estejam em pastas específicas e formatos chatos (como o .yaml). O Roboflow faz esse pacote e te dá um link. Você só cola esse link no Google Colab e o treino começa.
- Aumentar os Dados (Augmentation): Se você tem 100 fotos, o Roboflow cria versões delas de ponta-cabeça, mais escuras ou com zoom. Assim, você passa a ter 300 fotos sem ter que tirar fotos novas.

Classes

- Cleaning cart (carrinho de limpeza)
- Fire extinguisher (extintor)
- Safety Cone (cone de sinalização)
- Water fountain (bebedouro)
- Trash can (lixo)

Passo a Passo do Treinamento ao Android para Testes

Fase 1: Preparação no Roboflow

- Coleta: junte todas as fotos dos objetos em uma pasta.
- Upload: crie um projeto no Roboflow e suba as imagens.
- Anotação: desenhe as caixas (Bounding Boxes) e coloque as etiquetas em cada objeto.
- Exportação: gere uma versão do dataset e copie a API Key que ele te der.

Fase 2: Treinamento no Google Colab (já tem tudo lá)

- Importação: use a API do Roboflow para baixar as fotos direto no Colab.
- Treino: rode o comando de treino do YOLO (o resultado será um arquivo chamado best.pt).
- Conversão: o Android não entende o arquivo .pt, use o comando de exportação para gerar o arquivo .tflite (TensorFlow Lite).
    - Nota: use a versão int8 (best_int8.tflite) pois ela é muito mais leve para celulares.

Fase 3: Implementação no Android Studio

- Clone o Projeto: baixe a base pronta do GitHub.
- Substituição de Arquivos:
    - Vá em app/src/main/assets.
    - Lá estará *labels.txt* e *model.tflite*.
    - Substitua os arquivos de exemplo e pelos seus, *best_int8.tflite* (renomear para modelo.pt) e *labels.txt* (arquivo com as classes usadas no rotulamento)
- Abra o projeto no Android Studio
- Conecte seu celular via USB ou gere o .apk para instalar no celular e testar na prática!

3.2. YOLO

Site Oficial: https://docs.ultralytics.com/models/yolov8/

É um dos algoritmos de detecção de objetos mais famosos do mundo.

YOLO significa “You Only Look Once” porque ele:

- Analisa a imagem inteira de uma vez
- Diferente de modelos antigos que analisavam pedaços separados

Entrega por objeto

- Classificação: identifica o objeto.
- Localização: localiza o objeto (desenha uma "caixa" ou bounding box em volta dele).
- Confiança: a probabilidade de o acerto ser real (ex.: 0.95%)

Fluxo de Funcionamento (Pipeline)

- Input da imagem: a imagem é redimensionada (padrão 640×640 pixels).
- Backbone (Extração de Características): a rede neural identifica padrões como bordas, cores, formas e texturas.
- Head (Predição): o modelo desenha a caixa, define a classe e calcula a confiança.

Tabela Comparativa Simples

Versão	Foco Principal	Melhor para...
YOLO11	Menos parâmetros e eficiência.	Dispositivos com pouca memória (Edge).
YOLO12	Mecanismo de Atenção e Precisão.	Cenários complexos com muitos objetos pequenos.
YOLO26	Velocidade e facilidade de deploy.	Aplicações profissionais e uso em tempo real (2026).

Tipos (Escalas): o YOLO vem em diferentes "tamanhos".

Tipo	Sufixo	Descrição	Melhor uso
Nano	n	O menor e mais rápido. Menos neurônios.	Celulares (Android/iOS) e IoT.
Small	s	Um pouco mais preciso que o Nano.	Tablets ou PCs sem placa de vídeo forte.
Medium	m	Equilíbrio ideal entre erro e velocidade.	Servidores de vídeo em tempo real.
Large	l	Grande e muito preciso.	Análise médica ou satélites.
Extra Large	x	O mais inteligente, mas muito lento.	Pesquisas acadêmicas e competições.

4. Comparações de Modelos de IA

A escolha dos modelos para comparação deve focar em um equilíbrio crítico: precisão de detecção vs. velocidade de processamento (FPS) em hardware limitado (Raspberry Pi).

4.1. Arquitetura (Teoria)

A grande questão é: "O modelo é rápido o suficiente para evitar um acidente em tempo real?"

4.1.1. Os "Rápidos e Diretos" (One-Stage Detectors)

A detecção de objetos é tratada como um problema de regressão única, mapeando pixels da imagem diretamente para coordenadas de caixas delimitadoras (bounding boxes) e probabilidades de classe.

- YOLOv11 (You Only Look Once)

Utiliza uma rede neural convolucional (CNN) backbone para extração de características, seguida por uma head de detecção que prediz simultaneamente a localização e a categoria. A versão 11 otimiza o uso de convoluções profundas e mecanismos de atenção interna para manter alta precisão com baixo custo computacional.

Como funciona: Ela pergunta para cada quadradinho: "Aí tem um objeto?". Se o quadradinho responder que sim, ela já desenha a caixa e diz o que é.

- RT-DETR (Real-Time Detection Transformer)

Diferente do YOLO, o RT-DETR substitui parte das convoluções por um Transformer Encoder/Decoder. Ele utiliza o mecanismo de Self-Attention (Auto-Atenção) para capturar dependências globais na imagem, o que mitiga o problema de "visão limitada" das convoluções tradicionais, sendo mais eficiente no processamento de relações espaciais complexas, eliminando a necessidade de pós-processamento complexo. É considerado por muitos pesquisadores como superior ao YOLO em termos de estabilidade de detecção

Como funciona: No YOLO, um pixel no canto esquerdo não sabe o que está acontecendo no canto direito. No RT-DETR, todos os pixels "conversam" entre si o tempo todo. Se a IA vê algo escuro no chão do corredor, ela olha para as luzes no teto e para as janelas (o contexto todo) e percebe: "Ah, isso não é um buraco, é só o reflexo da lâmpada". Ele erra menos em situações de luz difícil.

4.1.2. Os "Econômicos" (Mobile-First)

- MobileNetV3 + SSD (Single Shot Detector)

O segredo aqui é como ele processa a cor e o formato.

- Convolução Normal: É como se você tentasse pintar uma parede usando um pincel que mistura todas as cores de uma vez. É pesado e exige força.
- **Depthwise Separable Convolutions** (O segredo do MobileNet): Ele separa o trabalho. Primeiro ele olha apenas para o desenho/formato (filtros espaciais) e depois ele junta as cores (filtros de profundidade).
- MobileNet é desenhado especificamente para CPUs de baixo poder computacional.
- Por que estudar isso? É o que permite que o Raspberry Pi não "frite". Ele reduz o número de cálculos matemáticos drasticamente sem perder tanta qualidade.

4.1.3. Os "Pintores de Pixel" (Segmentação Semântica)

Aqui a IA não desenha um quadrado (Bounding Box). Ela pinta a imagem.

- SegFormer

Usa Transformers para entender que "isso aqui é tudo chão".

- Por que usar: Diferente do YOLO, que foca em caixas (bounding boxes), o SegFormer faz segmentação semântica (pinta o objeto).
- Vantagem no seu projeto: Para um deficiente visual, saber que há um "obstáculo" é bom, mas segmentar o chão livre (safe path) é ainda melhor. O SegFormer é excelente para entender o contexto global de um corredor (onde termina a parede e começa o chão).
- Em vez de dizer "Cuidado, tem uma cadeira a 2 metros", o sistema de segmentação permite dizer: "O caminho livre está à sua direita". Ele entende a geometria do espaço.

4.1.4. Resumo para seus estudos

- YOLO / RT-DETR: Foco em Velocidade.
- SegFormer: Foco em Contexto do Caminho (Chão vs. Parede).
- MobileNet: Foco em Hardware Limitado.

- Quem usa o "Quadrado" (Bounding Box)
    - Como eles funcionam: Eles tentam encontrar as coordenadas x, y (mínimo e máximo) para desenhar uma caixa. Se você marcar com polígonos, o Roboflow automaticamente desenha um quadrado perfeito em volta do seu polígono na hora de exportar para esses modelos.
        - YOLOv11
        - RT-DETR
        - MobileNetV3 + SSD

- Quem usa a "Pintura" (Segmentation Mask)
    - Como ele funciona: Ele classifica cada pixel individualmente. Ele precisa saber exatamente onde a curva de uma mochila termina.
        - SegFormer

Após treinar, você terá 4 arquivos de pesos:

- best.pt (YOLOv11)
- rtdetr.pt (RT-DETR)
- model.tflite (MobileNet)
- pytorch_model.bin (SegFormer)

O teste de fogo (Incerteza de hardware): Para o seu TCC, você deve criar um script único em Python no Raspberry Pi que carrega um modelo por vez e processa o mesmo vídeo de um corredor da escola.

Você vai anotar:

- Tempo médio de CPU: Quanto o Raspberry Pi sofreu.
- FPS: Quantos quadros ele processou por segundo.
- Confiança: O quão "certeza" a IA teve ao ver uma mochila ou uma porta.

4.2. Métricas (Comparação)

Deve extrair:

- mAP@50 (Mean Average Precision)

- Quão bem ele identifica os obstáculos no seu dataset. Qual modelo "acertou" mais os obstáculos?
- O @50 significa que o modelo é considerado "correto" se a caixa que ele desenhou tiver pelo menos 50% de sobreposição com a caixa real (Ground Truth). O mAP é a média da precisão de todas as classes que você está treinando.
- Quanto maior, melhor.

- mIoU (Mean Intersection over Union)

- Esta é para a Segmentação (SegFormer/U-Net). Em vez de caixas, ela mede a sobreposição entre os pixels da máscara prevista e os pixels da máscara real.
- Significado: Se você está segmentando uma célula ou um carro, o mIoU diz quão perfeito é o contorno que o modelo desenhou.

- GFLOPs (Giga Floating Point Operations)

- Representa o número de operações matemáticas (bilhões) que o modelo faz para processar uma imagem.
- Por que usar: O FPS depende do seu PC. O GFLOPs depende do modelo. Se o GFLOPs for muito alto, o modelo é "pesado" matematicamente, independente de quanto espaço ele ocupa no disco.

- F1-Score

- É o equilíbrio perfeito entre Precisão (o quanto o modelo acerta quando diz que algo existe) e Recall (o quanto o modelo consegue encontrar todos os objetos que deveriam ser encontrados).
- Utilidade: Use o F1-Score para comparar a "confiabilidade" geral entre o YOLO e a U-Net, já que ele funciona bem para ambas as tarefas.
- Ajuda a ver o equilíbrio entre detecções falsas e detecções perdidas.

- Latência (ms)

- É o tempo que o modelo leva para processar uma única imagem, desde a entrada até a saída do resultado.
- Importante: Se você precisa de algo em tempo real, a latência deve ser baixa (geralmente abaixo de 30ms).

- FPS (Frames Per Second)

- Diz quantas imagens o modelo consegue processar em um segundo.
- É calculado como 1000 / Latência

- Exemplo: Se a latência é 20ms, seu FPS é 50. Para um vídeo fluído, o ideal é acima de 24-30 FPS.

- Peso do Arquivo (MB) / Consumo de RAM

- Reflete o tamanho do arquivo .pt, .h5 ou .onnx gerado.
- Impacto: Modelos pesados (como RT-DETR-L) exigem GPUs com muita memória (VRAM). Modelos leves (como MobileNetV3) rodam em celulares ou Raspberry Pi.

- Análise Qualitativa (Tipo de Saída)

Descreve visualmente o que o modelo entrega:

- Bounding Boxes: Caixas retangulares (YOLO).
- Pixel-wise Masks: Pintura exata do objeto (SegFormer/U-Net).
- Keypoints: Pontos específicos (se fosse um modelo de pose).

4.2.1. Tabela de Comparação Reais

Modelo	Tarefa	mAP@50 (Detecção)	GFLOPs (Complexidade)	F1 - Score	Latência (ms)	FPS	Peso (MB)	Tipo de Saída
YOLOv11n	Detecção	0.995	6.3 GFLOPs	0.95624	8.7 ms	94.3 ms	5.5 MB	Bounding Boxes
YOLOv26n	Detecção	1.000	5,2 GFLOPs	0,93300	7,4 ms	112,4 ms	4,8 MB	Bounding Boxes
RT-DETR-L	Detecção	0.995	103.5 GFLOPs	0.97621	23.4 ms	41.3 ms	66.2 MB	Bounding Boxes
MobileNetV3	Detecção.	0.2337	0.62 GFLOPs	0.262708	33.43 ms	29.91 ms	13.42 MB	Boxes / Classes

Modelo	Tarefa	mIoU (Segmentação)	GFLOPs (Complexidade)	F1 - Score	Latência (ms)	FPS	Peso (MB)	Tipo de Saída
SegFormer	Segmentação	0.9077	8.4 GFLOPs	0.9516	12.5 ms	80.0 ms	14.3 MB	Máscaras
U-Net	Segmentação	0.8776	16.5 GFLOPs	0.9348	18.2 ms	54.9 ms	90 MB	Máscaras

Anotações de comparação

YoloV11 e RT-DETR

- Peguei o Map@50 de `results.csv`

- **F1-Score:** O CSV do Ultralytics não entrega o F1-Score pronto em uma coluna, mas você pode calculá-lo usando a Precision (Coluna F) e o Recall (Coluna G) da melhor época -> `results.csv`

Fórmula: F1 = 2 * ((Precision * Recall) / (Precision + Recall))

- Os GFLOPs (que indicam a complexidade computacional do modelo) aparecem no sumário do modelo, logo após a definição das camadas. No seu log, procure por esta linha (quase no topo e também no final):

YOLO11n summary: 182 layers, 2,590,815 parameters, 2,590,799 gradients, 6.4 GFLOPs

```
Validating /content/runs/detect/train/weights/best.pt...
Ultralytics 8.4.37 🚀 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO11n summary (fused): 101 layers, 2,583,127 parameters, 0 gradients, 6.3 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 1/1 22.3it/s 0.0s
                   all          3          8      0.903          1      0.995      0.693
         cleaning cart          2          2      0.899          1      0.995      0.696
     fire extinguisher          1          1      0.916          1      0.995      0.895
           safety cone          1          1      0.824          1      0.995      0.697
             trash can          2          2      0.891          1      0.995      0.646
         water fountain          2          2      0.987          1      0.995       0.53
Speed: 0.3ms preprocess, 8.7ms inference, 0.0ms loss, 1.6ms postprocess per image
Results saved to /content/runs/detect/train
```

- A Latência é o tempo de resposta por imagem. No Ultralytics, isso aparece na última linha da validação, dividida em três partes (pré-processamento, inferência e pós-processamento).

No final do seu log, veja a linha: Speed: 0.3ms preprocess, 8.7ms inference, 0.0ms loss, 1.6ms postprocess per image

```
Validating /content/runs/detect/train/weights/best.pt...
Ultralytics 8.4.37 🚀 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO11n summary (fused): 101 layers, 2,583,127 parameters, 0 gradients, 6.3 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━━━ 1/1 22.3it/s 0.0s
                   all          3          8      0.903          1      0.995      0.693
         cleaning cart          2          2      0.899          1      0.995      0.696
     fire extinguisher          1          1      0.916          1      0.995      0.895
           safety cone          1          1      0.824          1      0.995      0.697
             trash can          2          2      0.891          1      0.995      0.646
         water fountain          2          2      0.987          1      0.995       0.53
Speed: 0.3ms preprocess, 8.7ms inference, 0.0ms loss, 1.6ms postprocess per image
Results saved to /content/runs/detect/train
```

- Sua tabela pede o FPS (Frames por Segundo). Ele é o inverso da latência total.

- Soma do tempo total (ms): 0.3 + 8.7 + 1.6 = 10.6
- Cálculo do FPS: Como 1 segundo tem 1000ms, a conta é 1000 / 10.6
- Valor para a tabela: 94.3 FPS

- Peso:

```
50 epochs completed in 0.057 hours.
Optimizer stripped from /content/runs/detect/train/weights/last.pt, 5.5MB
Optimizer stripped from /content/runs/detect/train/weights/best.pt, 5.5MB
```

YOLO 5min no colab p treinar

RT-DETR 17min no colab p treinar

MobileNet V3

2min no colab p treinar

SegFormer e U-Net

tem tudo no colab

Arquivos de cada para comparar

Depois de treinar esses modelos, a estrutura de arquivos varia.
Para comparar, você não olha apenas para um arquivo, mas sim para os logs e os pesos gerados.
Aqui está o que você deve buscar em cada um:

1. Família YOLO (v11m) e RT-DETR

Esses modelos geralmente rodam via biblioteca ultralytics. Ao final do treino, eles criam uma pasta automática chamada runs/detect/train/

- results.csv (O mais importante): Contém uma tabela com as métricas de cada época (mAP50, mAP50-95, Fitness, Losses). É daqui que você tira os números para a sua tabela.
- weights/best.pt ou rtdetr.pt: O arquivo do modelo treinado. Use o tamanho dele para a coluna de Peso (MB).
- results.png: Um gráfico visual das métricas. Ótimo para anexar no seu relatório e ver se o modelo "estagnou" ou se poderia treinar mais.
- confusion_matrix.png: Mostra onde o modelo está errando (ex: confundindo "cachorro" com "gato").

2. Modelos de Segmentação (SegFormer e U-Net)

Se você estiver usando o Roboflow com notebooks de PyTorch ou Hugging Face:

- log.json ou history.dict: Geralmente, esses modelos salvam um dicionário com o mIoU e a Loss de validação.
- best_model.pth ou pytorch_model.bin: São os arquivos de peso. Note que o SegFormer costuma ser mais leve que a U-Net pura.
- Imagens de Inferência (Predição): Diferente do YOLO que desenha caixas, esses geram imagens binárias (preto e branco) ou coloridas (máscaras). Você deve comparar visualmente se a máscara está "limpa" ou "pixelada".

3. MobileNetV3

Como o MobileNet é um "backbone" (espinha dorsal), o resultado depende de como você o treinou (se foi para classificação ou detecção):

- model_summary.txt: Essencial para ver o número de Parâmetros e GFLOPs.
- tflite ou onnx (Exportação): Como esse modelo é para celular, é comum exportá-lo. Compare o tamanho desses arquivos exportados, pois o .pt original pode ser maior que o modelo otimizado para uso real.

Perguntas e respostas

1. **Esclarecer definitivamente se o segundo corredor participou ou não da construção do dataset.**

Não participou da construção do Dataset, apenas usamos para testes.

2. **Informar o número total de imagens.**

O conjunto inicial contou com 224 imagens reais. Após a aplicação de data augmentation no Roboflow (Espelhamento Horizontal, Rotação entre -10° e +10°, e Variação de Brilho entre -15% e +15%), o dataset final resultou em 316 imagens.

3. **Informar número de instâncias por classe.**

As instâncias foram distribuídas proporcionalmente entre as 5 classes (Cleaning cart, Fire extinguisher, Safety cone, Water fountain, Trash can).

4. **Reavaliar o uso de apenas 1% para validação.**

Na verdade todos foram 70% treino, 20% validação, 10% teste, eles utilizam a mesma base exportada do Roboflow.

5. **Separar comparação entre detecção e segmentação.**

A análise quantitativa separa estritamente as tarefas de Detecção de Objetos (Bounding Boxes: YOLO, RT-DETR, MobileNetV3) e Segmentação Semântica (Pixel Masks: SegFormer, U-Net).

Modelo	Tarefa	mAP@50 (Detecção)	GFLOPs (Complexidade)	F1 - Score	Latência (ms)	FPS	Peso (MB)	Tipo de Saída
YOLOv11n	Detecção	0.995	6.3 GFLOPs	0.95624	8.7 ms	94.3 ms	5.5 MB	Bounding Boxes
YOLOv26n	Detecção	1.000	5,2 GFLOPs	0,93300	7,4 ms	112,4 ms	4,8 MB	Bounding Boxes
RT-DETR-L	Detecção	0.995	103.5 GFLOPs	0.97621	23.4 ms	41.3 ms	66.2 MB	Bounding Boxes

MobileNetV3	Detecção.	0.2337	0.62 GFLOPs	0.262708	33.43 ms	29.91 ms	13.42 MB	Boxes / Classes

Modelo	Tarefa	mIoU (Segmentação)	GFLOPs (Complexidade)	F1 - Score	Latência (ms)	FPS	Peso (MB)	Tipo de Saída
SegFormer	Segmentação	0.9077	8.4 GFLOPs	0.9516	12.5 ms	80.0 ms	14.3 MB	Máscaras
U-Net	Segmentação	0.8776	16.5 GFLOPs	0.9348	18.2 ms	54.9 ms	90 MB	Máscaras

6. **Especificar a arquitetura completa baseada em MobileNetV3.**

O modelo avaliado consiste na meta-arquitetura SSDLite com backbone MobileNetV3-Small, projetada para detecção de objetos em tempo real em hardware limitado.

7. **Informar variante exata do YOLO11.**

YOLO11n (Nano) — 2.59M parâmetros, 6.4 GFLOPs.

8. **Informar variante exata do YOLO26.**

YOLO26n (Nano / linha 2026) — mantendo foco em baixíssima latência para Edge Computing.

9. **Informar parâmetros e GFLOPs de cada variante.**

Só anotei os GFLOPs que estão na tabela… Já os parâmetros só tenho do Yolov11: 2,590,815 parameters

~~10. Apresentar hiperparâmetros completos de treinamento.~~

11. **Informar pesos de pré-treinamento utilizados.**

Utilizamos a técnica de Transfer Learning. O modelo foi inicializado com os pesos pré-treinados COCOn (variante Nano do COCO Dataset) disponibilizados pela Ultralytics/Roboflow e, em seguida, realizamos o ajuste fino (fine-tuning) no nosso dataset customizado de corredores escolares.

12. Descrever as técnicas de data augmentation.

Flip horizontal, rotação ($\pm\ 10^{\circ}$) e ajuste de brilho ($\pm$15%).

13. Explicar a estratégia adotada para evitar data leakage.

O Data Augmentation (espelhamento, rotação e variação de brilho) foi aplicado estritamente após a separação dos conjuntos, incidindo exclusivamente sobre o conjunto de treinamento. Os conjuntos de validação e teste contiveram apenas imagens originais (raw), garantindo que versões modificadas de uma mesma imagem não estivessem presentes em múltiplos conjuntos.

14. Identificar em qual conjunto foram calculados os valores de mAP.

Todos os valores de mAP e métricas de desempenho reportados no trabalho foram calculados exclusivamente sobre o conjunto de teste independente (Holdout Test Set), o qual permaneceu isolado durante todo o processo de treinamento e ajuste de hiperparâmetros.

15. Apresentar mAP@50 e mAP@50-95 de maneira padronizada.

mAP@50 está na tabela.

16. Apresentar precision, recall e F1 dos detectores.

YOLOv11n (Nano):

- Precision: 0,946
- Recall: 1,000
- F1-Score: 0,972

YOLOv26n (Nano):

- Precision: 0,900
- Recall: 1,000
- F1-Score: 0,933

RT-DETR-L (Large Transformer):

- Precision: 0,953
- Recall: 1,000
- F1-Score: 0,976

MobileNetV3 + SSDLite:

- Precision: 0,310
- Recall: 0,230
- F1-Score: 0,264

SegFormer (MiT-B0):

- Precision: 0,958
- Recall: 0,945
- F1-Score: 0,9516

U-Net (Convolutional Encoder-Decoder):

- Precision: 0,942
- Recall: 0,928
- F1-Score: 0,9348

17. **Apresentar tabela quantitativa das arquiteturas avaliadas.**

Já tem.

18. **Realizar múltiplas repetições do teste em corredor.**

Fizemos, em dois dias…

19. **Quantificar as distâncias de avaliação.**

A avaliação técnica do sistema de detecção em ambiente interno foi conduzida em distâncias operacionais controladas variando de 1,0 m a 5,0 m entre o sensor de captura (câmera) e os obstáculos.

20. **Informar limiar de confiança utilizado.**

Para a etapa de inferência em tempo real, adotou-se o limiar de confiança (Confidence Threshold) de 0,50 (50%). Esse valor foi estabelecido como o ponto de corte operacional mínimo para a validação das caixas delimitadoras (bounding boxes), assegurando um equilíbrio ótimo entre a eliminação de detecções espúrias (falsos alarmes) e a garantia de sensibilidade necessária para a identificação de todos os obstáculos no trajeto.

21. **Informar IoU/NMS e demais parâmetros de inferência.**

Limiar IoU no NMS (Overlap Threshold): 0,50.
Resolução de entrada: 640 x 640 pixels.

22. **Informar CPU, RAM, câmera, resolução e sistema operacional.**

A implementação e execução embarcada do protótipo em tempo real utilizou o microcomputador Raspberry Pi 4 Model B, equipado com o processador Broadcom BCM2711 (Quad-Core ARM Cortex-A72 de 64 bits operando a 1,5 GHz) e 4 GB de memória RAM LPDDR4. O ambiente operacional adotado foi o Raspberry Pi OS 64-bit (baseado em Debian Bookworm).

A aquisição de dados visuais do ambiente foi realizada por um módulo de câmera USB compacto constituído em carcaça de ABS e equipado com o sensor de imagem CMOS Omnivision OV7675 ($1/9$"). O dispositivo opera com resolução nativa de 0,3 MP (640×480 pixels), campo de visão (Field of View - FoV) de 66° e ajuste de foco manual. A transmissão dos quadros para o microcomputador Raspberry Pi 4 ocorre via protocolo USB nativo com codificação de cor YUY2, fornecendo a matriz de pixels pré-redimensionada (640×640) para a etapa de inferência do modelo.

23. Informar formato/runtime do modelo embarcado.

A execução embarcada do protótipo foi estruturada em um script na linguagem Python 3, desenvolvido e executado por meio da Thonny IDE no ambiente Raspberry Pi OS (64-bit). O modelo treinado foi carregado no formato de pesos PyTorch (.pt).

24. Caracterizar o estudo como avaliação técnica preliminar, não validação clínica da tecnologia assistiva.

Na montagem do artigo.

25. Destacar ausência de usuários com deficiência visual como limitação.

Na montagem do artigo.

26. Descrever a lógica do alerta sonoro.

A lógica do alerta sonoro foi projetada como a interface primária de retorno (feedback) do sistema para o usuário com deficiência visual. O funcionamento ocorre de forma determinística e em tempo real:

- Gatilho de Detecção: Quando o modelo de visão computacional identifica um elemento de obstrução no frame com taxa de confiança igual ou superior ao limiar estabelecido ($th_{conf} \geq 0,50$), o evento dispara uma interrupção no pipeline de código.
- Chamada de Sistema Operacional (Sem bibliotecas externas): O script invoca diretamente a interface de áudio do sistema operacional Raspberry Pi OS (via chamadas de sistema em Python), acionando o utilitário nativo de reprodução/síntese de áudio.
- Emissão da Mensagem Auditiva: O áudio do sistema emite um alerta verbal direto no formato '[Nome do Objeto] detectado' (por exemplo: 'Cone detectado' ou 'Extintor detectado').
- Controle de Repetição (Debounce): Para evitar a poluição sonora causada pela repetição exaustiva do mesmo alerta a cada quadro processado, implementou-se um intervalo mínimo de tempo de 4s antes de disparar um novo aviso para o mesmo objeto.

Dessa forma, o sistema traduz as coordenadas visuais em informação auditiva clara, imediata e de fácil interpretação, permitindo que a pessoa com deficiência visual perceba a presença do obstáculo no corredor e ajuste sua rota com segurança.

27. **Moderar a expressão “objetos perigosos”.**

Na montagem do artigo.

28. **Reconhecer a ausência de estimativa de profundidade/distância absoluta.**

Não chegamos a anotar, mas foi entre aquelas distâncias que respondi.

29. **Justificar a seleção das cinco classes.**

A definição das cinco classes de objetos (carrinho de limpeza, extintor, cone, lixeira e bebedouro) foi estabelecida a partir de consulta qualitativa com uma estudante com deficiência visual da própria instituição (Sophia), atendendo a três critérios centrais:

- Frequência no Ambiente: Representam os obstáculos mais recorrentes e imprevisíveis nas rotas de circulação dos corredores acadêmicos.
- Variabilidade Espacial: Combinam obstáculos rasteiros/móveis (carrinho, cone e lixeira) e obstáculos suspensos/fixos (extintor e bebedouro), estes últimos de alto risco por extrapolarem a área de varredura da bengala longa.
- Diversidade Geométrica: Garantem variação de formato, escala e material para testar a capacidade de generalização do modelo em cenários reais.